Colin Fergusson
3664465
CS4745
Lab 4

I was unable to get my code to run due to the lack of MPI on the lab machines, I had assumed that it would've been added to them by the end of the week, but it hasn't been. I have still done the lab to the best of my ability, just without actually running my code so it almost certainly does not work.

Sizes Tested:
The sizes I used for the ping pong function were 1, 8, 64, 512, and 4096 bytes

Average RTT and estimated one-way latency:
I do not have the average RTT results of my code since I couldn't run it, but if it were to run it should print out the proper time for each size tested as well as the one way latency.

How message size is controlled:
I controlled the message size by running the ping pong function 5 seperate times, each one having a different message size as the input. The message size stays constant during the execution of the function.

How correctness is verified:
In the ping pong function correctness is verified through a tag that is generated based on the current iteration the function is on. That tag is given to both processes and will always be the same for both sending and recieving information.

In the ring function I have two tags set as constants, one for sending and one for recieving. Each process uses the same tag for sending and the same tag for recieving.

Why your implementation avoids deadlock:
For the ping pong function deadlock is avoided because MPI_Recv is a block and each process waits for the message to be sent before moving past it. But because each process does an MPI_Send before getting to the MPI_Recv neither of them will ever get stuck since there will always be information to recieve once getting to the MPI_Recv.

For the ring function deadlock is avoided because of the use of MPI_Sendrecv. Every process is sending a message and then awaiting a message at the same time, so as they are waiting for a message after sending it out all the other processes will have either already sent a message or are currently sending it which means that none of the processes will get stuck waiting.